

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 2

Roll no:06 PRN No: 12413462 Name of Student: Purva Patil

Subject Name : Deep Learning

Problem Statement :

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm :

1. Import the required Python libraries.
2. Load the Wine dataset from Scikit-learn.
3. Convert the dataset into a Pandas DataFrame.
4. Explore the dataset using summary statistics and check for missing values.
5. Visualize the class distribution and feature correlations.
6. Split the dataset into training and testing sets.
7. Standardize the feature values using StandardScaler.
8. Create an MLP classifier with two hidden layers.
9. Train the model using the training data.
10. Predict the class labels for the test data.
11. Evaluate the model using accuracy, confusion matrix, and classification report.
12. Visualize the confusion matrix.
13. Analyze the results and conclude the performance of the MLP classifier.

Code :

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

iris = load_iris()

X = iris.data
y = iris.target

x_train, x_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

```

```

def create_model(lr):
    model = Sequential([
        Dense(128, activation='relu', input_shape=(4,)),
        Dense(64, activation='relu'),
        Dense(3, activation='softmax')
    ])

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

```

```

learning_rates = [0.1, 0.01, 0.001]

results = []

for lr in learning_rates:

    model = create_model(lr)

    model.fit(

```

```

        x_train,
        y_train,
        epochs=20,
        batch_size=16,
        verbose=0
    )

    loss, acc = model.evaluate(
        x_test,
        y_test,
        verbose=0
    )

    results.append([lr, acc])

df = pd.DataFrame(results,
                   columns=["Learning Rate", "Accuracy"])

print(df)

plt.figure(figsize=(6,4))
plt.plot(df["Learning Rate"],
         df["Accuracy"],
         marker='o')

plt.xscale("log")
plt.xlabel("Learning Rate")
plt.ylabel("Accuracy")
plt.title("Learning Rate vs Accuracy")
plt.grid(True)
plt.show()

```

```

epochs_list = [5,10,20]

res = []

for ep in epochs_list:

    model = create_model(0.001)

    history = model.fit(
        x_train,
        y_train,
        validation_split=0.2,
        epochs=ep,
        batch_size=16,
        verbose=0
    )

```

```

        loss, acc = model.evaluate(
            x_test,
            y_test,
            verbose=0
        )

        res.append([ep, acc])

edf = pd.DataFrame(
    res,
    columns=["Epochs", "Accuracy"]
)

print(edf)

plt.figure(figsize=(6,4))
plt.plot(
    edf["Epochs"],
    edf["Accuracy"],
    marker='o'
)

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Epochs vs Accuracy")
plt.grid(True)
plt.show()

model = create_model(0.001)

history = model.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=16,
    verbose=1
)

```

```

plt.figure(figsize=(6,4))
plt.plot(history.history["accuracy"])
plt.plot(history.history["val_accuracy"])

plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.grid(True)
plt.show()

```

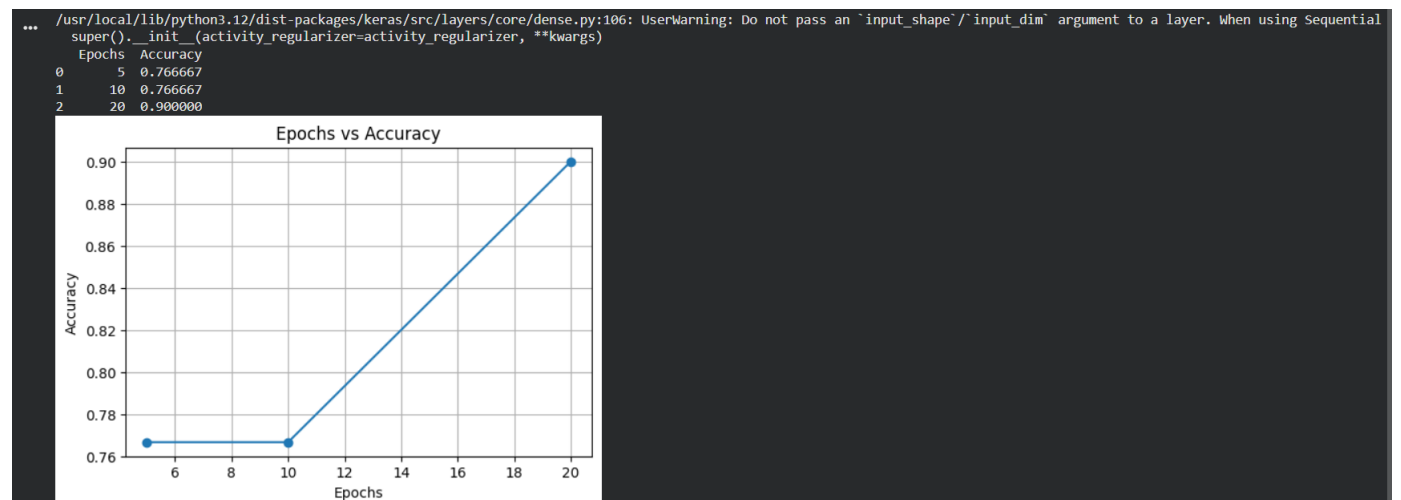
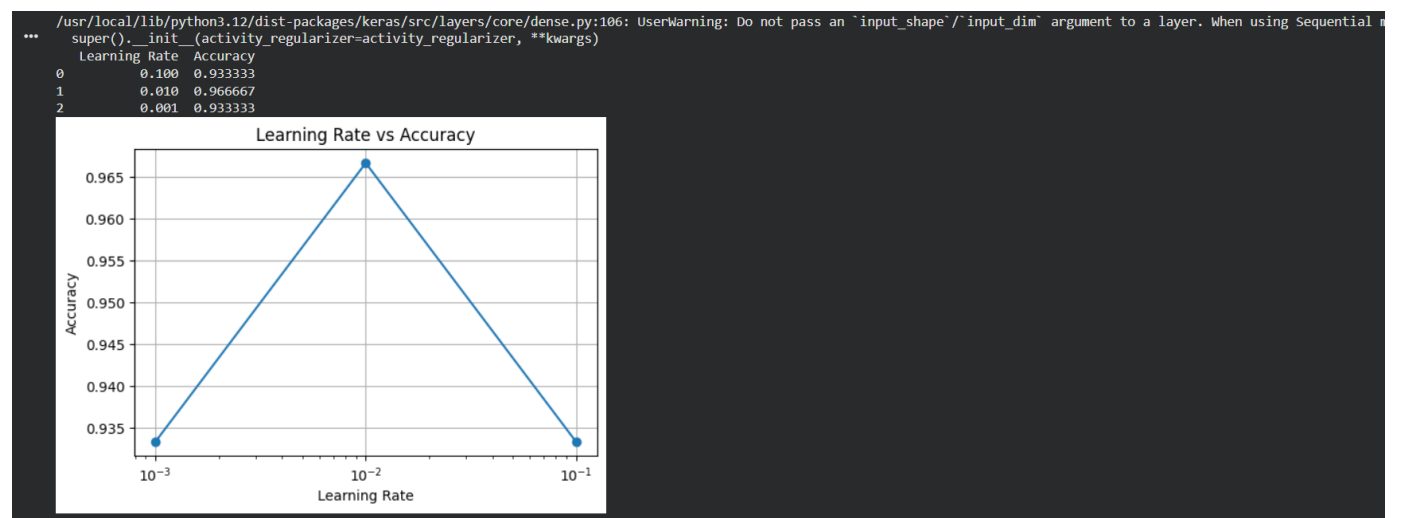
```

loss, accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("\nTest Loss :", loss)
print("Test Accuracy :", accuracy)

```

Output :



```

Epoch 1/20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using the Functional API, you should only pass an `input_shape` argument to the first layer in a stack.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
6/6 ━━━━━━━━━━━ 2s 77ms/step - accuracy: 0.3958 - loss: 1.0857 - val_accuracy: 0.8333 - val_loss: 0.9391
Epoch 2/20
6/6 ━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.7708 - loss: 0.8655 - val_accuracy: 0.7500 - val_loss: 0.7859
Epoch 3/20
6/6 ━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.8438 - loss: 0.7140 - val_accuracy: 0.7500 - val_loss: 0.6745
Epoch 4/20
6/6 ━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.8333 - loss: 0.5984 - val_accuracy: 0.7917 - val_loss: 0.5976
Epoch 5/20
6/6 ━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.8438 - loss: 0.5106 - val_accuracy: 0.7917 - val_loss: 0.5400
Epoch 6/20
6/6 ━━━━━━━━━━━ 0s 35ms/step - accuracy: 0.8542 - loss: 0.4471 - val_accuracy: 0.7917 - val_loss: 0.4893
Epoch 7/20
6/6 ━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.8542 - loss: 0.3983 - val_accuracy: 0.7917 - val_loss: 0.4494
Epoch 8/20
6/6 ━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.8542 - loss: 0.3614 - val_accuracy: 0.8333 - val_loss: 0.4155
Epoch 9/20
6/6 ━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.8542 - loss: 0.3317 - val_accuracy: 0.8333 - val_loss: 0.3824
Epoch 10/20
6/6 ━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.8750 - loss: 0.3091 - val_accuracy: 0.8750 - val_loss: 0.3463
Epoch 11/20
6/6 ━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.8750 - loss: 0.2870 - val_accuracy: 0.8750 - val_loss: 0.3254
Epoch 12/20
6/6 ━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8750 - loss: 0.2692 - val_accuracy: 0.8750 - val_loss: 0.2980
Epoch 13/20
6/6 ━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8854 - loss: 0.2541 - val_accuracy: 0.8750 - val_loss: 0.2684
Epoch 14/20
6/6 ━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8958 - loss: 0.2376 - val_accuracy: 0.9167 - val_loss: 0.2513
Epoch 15/20

```

```

Epoch 16/20
6/6 ━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9062 - loss: 0.2156 - val_accuracy: 0.9167 - val_loss: 0.2120
Epoch 17/20
6/6 ━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9167 - loss: 0.2027 - val_accuracy: 0.9167 - val_loss: 0.1996
Epoch 18/20
6/6 ━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9167 - loss: 0.1916 - val_accuracy: 0.9167 - val_loss: 0.1840
Epoch 19/20
6/6 ━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9271 - loss: 0.1826 - val_accuracy: 1.0000 - val_loss: 0.1675
Epoch 20/20
6/6 ━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9271 - loss: 0.1726 - val_accuracy: 1.0000 - val_loss: 0.1599

```

